

CSE 3102: SOFTWARE ENGINEERING

3rd Year 1st Semester Final Examination 2024

Yearly Questions & Solutions

Question

- 1(a) What is software process model? Why it is necessary? (3)
- 1(b) Discuss the main drawbacks of Waterfall model and mention the cases where this model is more preferable. (3)
- 1(c) Explain how verification and validation are handled in V model. Provide one scenario where the V-Model is highly effective and one where it may not be suitable. (4)
- 1(d) Describe the four key phases of the Spiral Model. Explain how this model handles risk assessment and mitigation. (4)

Solution

1(a): Software Process Model

A **software process model** is a structured framework that defines the **activities, sequence, deliverables, and controls** used to develop, deliver, and maintain software (e.g., Waterfall, V-Model, Spiral, RUP, Agile).

Why necessary:

- Provides **predictability** (planning, scheduling, milestones).
- Improves **quality** through defined reviews and testing steps.
- Supports **management** (cost estimation, tracking, risk control).
- Ensures **standardization** and better team communication.

Solution

1(b): Waterfall Drawbacks + When Preferable

Drawbacks:

- Rigid and **sequential**; difficult to go back after finishing a phase.
- **Late testing/feedback**: errors discovered late become expensive to fix.
- Poor for **changing requirements**; not flexible.
- Working software appears late, so **customer visibility** is low.

Preferable when:

- Requirements are **stable, clear, and fixed**.
- Technology is well understood (low uncertainty).

- Strong documentation and compliance are needed (e.g., some regulated projects).

Solution

1(c): Verification vs Validation in V-Model

In the **V-Model**, development phases on the left map to corresponding testing phases on the right.

Verification (“Are we building the product right?”):

- Reviews/inspections of artifacts: requirements, design, code.
- Ensures correctness of outputs of each phase **before** moving forward.

Validation (“Are we building the right product?”):

- Executes tests to confirm software meets user needs.
- Mapping example: Requirements ↔ Acceptance testing, Architecture ↔ System testing, Detailed design ↔ Integration testing, Code ↔ Unit testing.

Highly effective scenario: Safety/mission-critical systems with fixed requirements (e.g., avionics modules, medical device software).

Not suitable scenario: Requirements frequently change (startup product, exploratory UI-heavy apps).

Solution

1(d): Spiral Model Phases + Risk Handling

The Spiral Model is iterative and risk-driven. Each cycle contains four phases:

1. **Objective Setting:** define goals, alternatives, constraints.
2. **Risk Assessment & Reduction:** identify risks; prototype/simulate to reduce them.
3. **Development & Validation:** build and test the next increment.
4. **Planning:** plan next iteration (cost, schedule, scope).

Risk mitigation: risks are analyzed early in each cycle; high-risk parts are prototyped first, so major failures are detected early.

Question 2

Question

- 2(a) What is a Rational Unified Process (RUP) model? Define the four key phases of RUP (Inception, Elaboration, Construction, Transition). (5)
- 2(b) Explain how RUP's disciplines (e.g., Requirements, Design, Implementation) align or conflict with Agile practices. (3)
- 2(c) What is the difference between the Prototype model and RUP model in terms of deliverables in each iteration? (2)
- 2(d) Define coupling and cohesion in case of object oriented programs? What degrees of coupling and cohesion are desirable? Explain your answer. (4)

Solution

2(a): RUP and its Phases

RUP is an iterative, incremental, use-case driven, architecture-centric process framework.

Phases:

- **Inception:** scope, business case, feasibility, high-level requirements.
- **Elaboration:** baseline architecture, mitigate major risks, detailed requirements.
- **Construction:** iterative implementation, testing, integration, working increments.
- **Transition:** deployment, user training, fixing release issues, product delivery.

Solution

2(b): RUP Disciplines vs Agile

Alignments:

- Iterative development and incremental delivery.
- Continuous testing and integration are compatible with Agile.

Conflicts (possible):

- RUP can be documentation-heavy and process-heavy if not tailored.
- Agile prefers lightweight artifacts and faster feedback loops.

Solution

2(c): Prototype vs RUP Deliverables

Prototype model: iteration output is usually a **prototype** (often disposable) to clarify requirements.

RUP: each iteration produces **production-quality increments** plus evolving artifacts (use cases, architecture, test cases).

Solution

2(d): Coupling and Cohesion

Cohesion: how strongly related responsibilities inside a class/module are.

Coupling: how strongly one class/module depends on another.

Desirable: High cohesion and low coupling.

Reason: high cohesion improves clarity/reuse; low coupling improves flexibility, testability, and ease of change.

Question 3

Question

3(a) Which of the following scenarios represent valid use cases? Explain your answer. (4)

i. Negotiate a supplier contract ii. Handle returns iii. Log in iv. Move piece on game Board

3(b) From the use case “Process Sale” described below, answer (RUP style): (1+3+3)

- Identify the primary actor.
- Main success scenario.
- Two extensions (failure scenarios) and handling.

“Customer arrives at a checkout with items to purchase. The cashier uses the POS system to record each item. The system presents a running total and lineitem details. The customer enters payment information, which the system validates and records. The system updates inventory. The customer receives a receipt from the system and then leaves with the items.”

3(c) Identify a reusable subfunction that should be «include»d. Also define an extension point and conditions under which the «extend» case triggers. (3)

Solution

3(a): Valid Use Cases

A use case describes an **actor goal** achieved through interactions with the system.

- (i) **Negotiate a supplier contract:** Typically a business process; valid if the system supports negotiation workflow. Otherwise it may be outside the system boundary.
- (ii) **Handle returns: Valid.** Clear user goal with system interactions.
- (iii) **Log in: Valid** (common supporting use case).
- (iv) **Move piece on game board: Valid** if it is a player goal supported by the game system (e.g., “Make a move”).

Solution

3(b): Use Case — Process Sale (RUP Style)

i. **Primary Actor: Cashier** (operates POS system).

ii. **Main Success Scenario (example):**

1. Customer arrives with items.
2. Cashier starts a new sale in POS.
3. Cashier records/scans each item.
4. System displays line items and running total.
5. Customer provides payment information.
6. System validates payment and records transaction.
7. System updates inventory.
8. System prints/provides receipt.
9. Customer leaves with items.

iii. **Two Extensions (Failure Cases):**

- **E1: Payment authorization fails** (insufficient funds / invalid card).
Handling: system shows failure reason; cashier requests another payment method or cancels sale.
- **E2: Item barcode not found / invalid item.**
Handling: system alerts cashier; cashier manually enters item code or calls supervisor; continue scanning.

Solution

3(c): Include and Extend

Reusable «include» subfunction: Validate Payment (can be reused in sale, refund, bill payment).

Extension point: After “Validate Payment”.

«extend» use case: Apply Discount / Loyalty Offer.

Trigger condition: customer has loyalty card / promo code / eligible discount; the extension inserts discount calculation before finalizing payment.

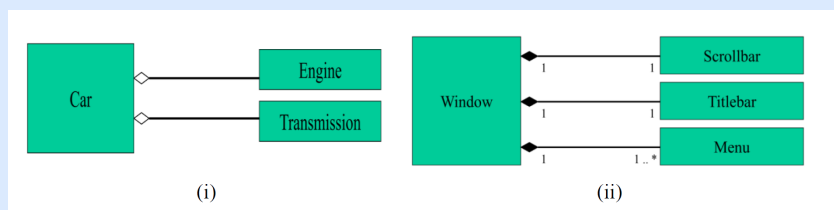
Question 4

Question

4(a) From the scenario described, design a domain model (entities, relationships, key attributes) and justify design choices. (7)

“A patient schedules an appointment with a doctor through a clinic’s online system. The system checks the doctor’s availability and confirms the appointment. On the day of the visit, the receptionist verifies the patient’s details and updates their medical record. The doctor examines the patient, diagnoses the condition, and prescribes medication. The system records the diagnosis, updates the treatment history, and generates a prescription. The patient receives the prescription and proceeds to the pharmacy for medication pickup.”

4(b) Difference between Aggregation and Composition in class diagrams? Explain given diagrams (i) and (ii). (5)



4(c) Primary differences between class diagram and domain model diagram (level of abstraction). (2)

Solution

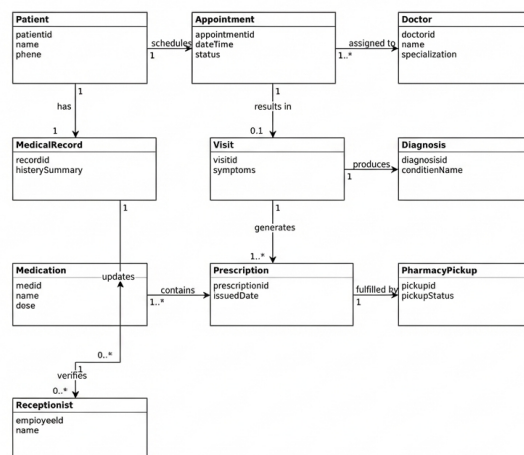
4(a): Domain Model (Clinic Appointment Scenario)

Key entities (with example attributes):

- **Patient**(*patientId*, name, phone, address)
- **Doctor**(*doctorId*, name, specialization)
- **Appointment**(*appointmentId*, dateTime, status)
- **MedicalRecord**(*recordId*, allergies, historySummary)
- **Visit/Consultation**(*visitId*, symptoms, notes)
- **Diagnosis**(*diagnosisId*, conditionName, severity)
- **Prescription**(*prescriptionId*, issuedDate)
- **Medication**(*medId*, name, dose, duration)
- **PharmacyPickup**(*pickupId*, pickupStatus)
- **Receptionist**(*employeeId*, name)

Relationships:

- A **Patient** can schedule multiple **Appointments**, but each appointment is associated with exactly one patient.
- A **Doctor** can have multiple appointments, while each appointment is assigned to one doctor.
- An **Appointment** may result in at most one **Visit**, representing the actual consultation.
- Each **Visit** produces exactly one **Diagnosis**.
- Each **Patient** maintains a single **MedicalRecord**, which is updated across visits.
- A **Diagnosis** can generate one or more **Prescriptions**.
- Each **Prescription** consists of one or more **Medications** (line items).
- A **Prescription** may optionally be linked to a **PharmacyPickup** for medication collection.
- The **Receptionist** verifies patient details and updates the medical record (association relationship).



Justification (clarity & scalability):

- **Appointment** is separated from **Visit** to clearly distinguish scheduling from the actual medical examination.
- **MedicalRecord** is modeled independently to preserve long-term patient history across multiple visits.
- Keeping **Diagnosis** and **Prescription** as separate entities supports auditing, history tracking, and multiple treatments over time.
- Modeling **Medication** separately allows a prescription to contain multiple drugs with different dosages and durations.
- **PharmacyPickup** is separated to represent post-prescription workflow without overloading prescription responsibilities.

Solution

4(b): Aggregation vs Composition + Diagram Explanation

Aggregation (hollow diamond): “has-a” weak ownership. Parts can exist independently.

Composition (filled diamond): strong ownership. Part’s lifecycle depends on whole.

Diagram (i): Car – Engine/Transmission (Aggregation)

Car *aggregates* Engine and Transmission: they can exist separately and can be replaced.

Diagram (ii): Window – Scrollbar/Titlebar/Menu (Composition)

Window *composes* UI parts: if the Window is destroyed, its UI components are also destroyed.

Multiplicity shows Window has exactly one Scrollbar, one Titlebar, and one-or-more Menu items (as indicated in diagram).

Solution

4(c): Class Diagram vs Domain Model Diagram

Domain Model: conceptual, real-world entities and relationships; **technology-independent**; focuses on nouns and business meaning.

Class Diagram: software design view; includes methods, visibility, types, inheritance, interfaces; **implementation-oriented** and more detailed.

Question 5

Question

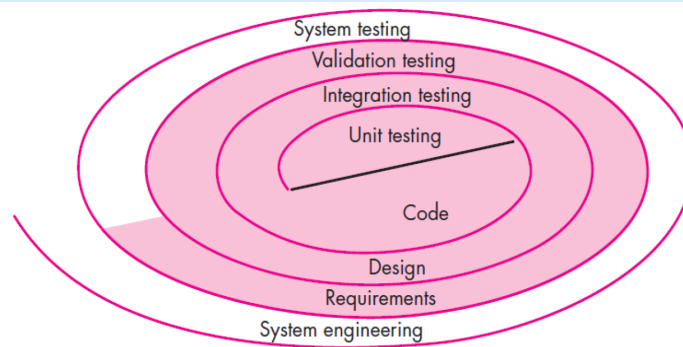
5(a) Draw the sequence diagram from the given code snippet. (6)

```
public class PaymentSystem {
    public void processPayment(Order order) {
        if (order.isValid()) {
            while (!order.isPaid()) {
                order.charge();
            }
            order.ship();
        } else {
            order.reject();
        }
    }
}
```

5(b) Explain UAT (User acceptance testing). Why regression testing is done? (2+2)

5(c) Mention key differences between static and dynamic analysis. (2)

5(d) Explain the given figure in terms of software testing activity. (2)

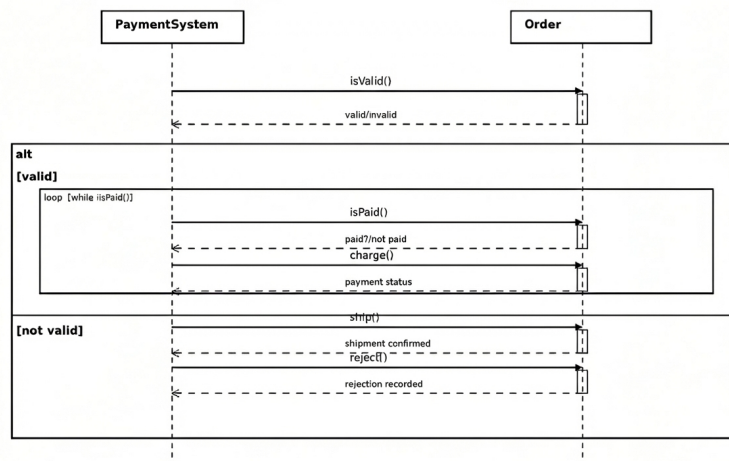


Solution

5(a): Sequence Diagram (Textual/Message Form)

Participants: **PaymentSystem** and **Order**. The sequence is:

1. PaymentSystem → Order: isValid()
2. **alt** [valid]
 - **loop** [while !isPaid()]
 - PaymentSystem → Order: isPaid()
 - PaymentSystem → Order: charge()
 - PaymentSystem → Order: ship()
3. **else** [not valid]
 - PaymentSystem → Order: reject()



Solution

5(b): UAT and Regression Testing

UAT (User Acceptance Testing): Testing performed by end users/customers to confirm the system meets business needs and acceptance criteria before release.

Regression testing: Re-testing after changes (bug fixes/new features) to ensure existing functionality did not break.

Solution

5(c): Static vs Dynamic Analysis

Static analysis: examines code/artifacts **without execution** (linting, review, dataflow checks).

Dynamic analysis: evaluates software **during execution** (runtime behavior, memory leaks, performance).

Solution

5(d): Explanation of the Figure (Testing Levels)

The figure shows testing as expanding levels around the code:

- **Unit testing** validates individual units/modules near the code.
- **Integration testing** checks interactions between modules.
- **Validation testing** confirms requirements/user needs.
- **System testing** evaluates the complete system in its environment.

As we move outward, scope increases and tests align with higher-level artifacts (design/requirements/system engineering).

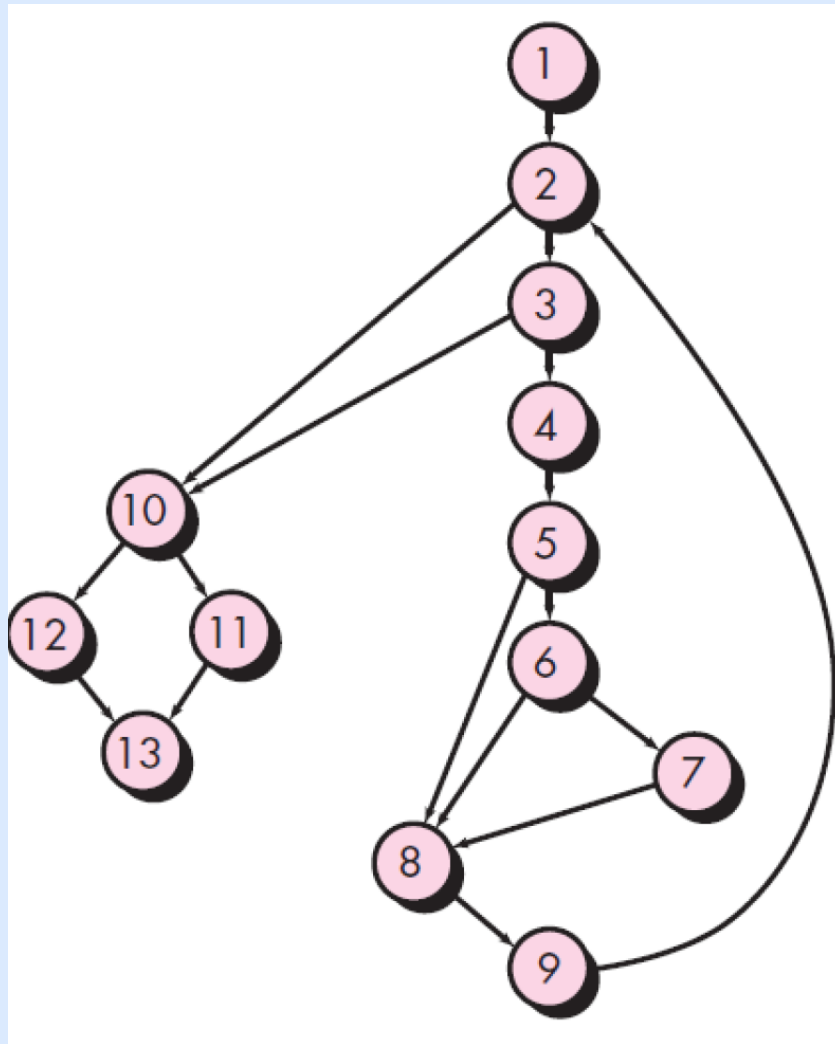
Question 6

Question

6(a) Explain the primary purpose of using CFGs in program analysis. What does a basic block represent in a CFG? (4)

6(b) How are decision points (e.g., if conditions) represented in a CFG? Illustrate with a simple code snippet. (2)

6(c) From the flow graph: (i) basis set of linearly independent paths (ii) cyclomatic complexity. (5)



6(d) Explain stubs and drivers in unit testing. Difference between top-down and bottom-up integration testing. (3)

Solution

6(a): CFG Purpose and Basic Block

CFG (Control Flow Graph) represents all possible execution paths of a program. It is used for:

- path-based testing (basis path testing),
- complexity measurement (cyclomatic complexity),
- compiler optimizations and static analysis.

A **basic block** is a maximal sequence of straight-line statements with **one entry and one exit** (no jumps inside).

Solution

6(b): Decision Points in CFG

Decision points create **branch nodes** with multiple outgoing edges.

```
if (x > 0) {  
    a = 1;  
} else {  
    a = -1;  
}
```

In CFG: the node for condition $x > 0$ has two outgoing edges: **True** branch to the $a = 1$ block and **False** branch to the $a = -1$ block.

Solution

6(c): Basis Paths and Cyclomatic Complexity

From the given flow graph (nodes 1–13), using:

$$V(G) = E - N + 2$$

where $N = 13$ nodes and $E = 17$ edges (counting all directed connections), we get:

$$V(G) = 17 - 13 + 2 = 6$$

So, **cyclomatic complexity = 6**, and we need **6 linearly independent basis paths**.

One valid basis set:

1. $1 \rightarrow 2 \rightarrow 10 \rightarrow 11 \rightarrow 13$
2. $1 \rightarrow 2 \rightarrow 10 \rightarrow 12 \rightarrow 13$
3. $1 \rightarrow 2 \rightarrow 3 \rightarrow 10 \rightarrow 11 \rightarrow 13$
4. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 8 \rightarrow 9 \rightarrow 2 \rightarrow 10 \rightarrow 11 \rightarrow 13$
5. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 8 \rightarrow 9 \rightarrow 2 \rightarrow 10 \rightarrow 11 \rightarrow 13$
6. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 2 \rightarrow 10 \rightarrow 11 \rightarrow 13$

Solution

6(d): Stubs, Drivers, Top-down vs Bottom-up

Driver: a dummy calling program that calls the module under test (used when higher-level module not ready).

Stub: a dummy called module that returns canned results (used when lower-level module not ready).

Top-down integration: integrate from main/control modules downward; uses many **stubs** early.

Bottom-up integration: integrate from low-level utilities upward; uses many **drivers** early.

Question 7

Question

7(a) Why software design patterns are necessary? Explain with an example. (3)

7(b) From the code snippet, what changes should be made to make it work as a singleton pattern. Justify. (5)

```
public class SingletonExample {  
  
    private static SingletonExample ref = null;  
  
    public static SingletonExample getInstance() {  
        if (ref == null) {  
            ref = new SingletonExample();  
        }  
        return ref;  
    }  
  
    private SingletonExample() { // ... more code }  
}
```

7(c) Use the idea of Factory pattern in the following scenarios: (6)

- (i) A PizzaStore uses a SimplePizzaFactory to create pizzas (CheesePizza, PepperoniPizza, VeggiePizza). Write Java code for SimplePizzaFactory.createPizza(type) that returns the appropriate Pizza subclass.
- (ii) PizzaStore is now abstract, with subclasses (NYStylePizzaStore, ChicagoStylePizzaStore) overriding createPizza() to produce regional variants. Write the abstract PizzaStore class with the factory method and orderPizza(). Implement createPizza() in NYStylePizzaStore to return NYStyleCheesePizza.

Solution

7(a): Why Design Patterns? + Example

Design patterns are necessary because they provide **proven reusable solutions** to recurring design problems, improving maintainability, flexibility, and communication.

Example: Observer pattern in GUI event handling: button (subject) notifies listeners (observers) when clicked, without tightly coupling button to specific listener classes.

Solution

7(b): Fixing Singleton (Thread-safe)

The given code represents a **lazy-initialized Singleton**. In its original form, such a singleton may fail in a multi-threaded environment due to a **race condition**. This issue is resolved here by synchronizing the `getInstance()` method.

```
public class SingletonExample {
```

```

private static SingletonExample ref = null;

public static synchronized SingletonExample getInstance() {
    if (ref == null) {
        ref = new SingletonExample();
    }
    return ref;
}

private SingletonExample() { }
}

```

Justification:

- The `synchronized` keyword ensures that only **one thread** can execute the `getInstance()` method at a time.
- This guarantees that **only one instance** of the class is created, even in a multi-threaded environment.
- Although this approach introduces synchronization overhead on every method call, it provides a **simple and correct thread-safe Singleton implementation**.

Solution

7(c): Factory Pattern (Pizza Scenarios)

(i) Simple Factory:

```

abstract class Pizza { }

class CheesePizza extends Pizza { }
class PepperoniPizza extends Pizza { }
class VeggiePizza extends Pizza { }

class SimplePizzaFactory {
    public Pizza createPizza(String type) {
        if (type == null) return null;
        switch (type.toLowerCase()) {
            case "cheese": return new CheesePizza();
            case "pepperoni": return new PepperoniPizza();
            case "veggie": return new VeggiePizza();
            default: return null;
        }
    }
}

```

(ii) Factory Method:

```

abstract class PizzaStore {

    public Pizza orderPizza(String type) {
        Pizza pizza = createPizza(type);
        // (Optional steps) prepare/bake/cut/box
    }
}

```

```
        return pizza;
    }

    protected abstract Pizza createPizza(String type);
}

class NYStyleCheesePizza extends Pizza { }

class NYStylePizzaStore extends PizzaStore {
    @Override
    protected Pizza createPizza(String type) {
        if (type == null) return null;

        if ("cheese".equalsIgnoreCase(type)) {
            return new NYStyleCheesePizza();
        }
        return null;
    }
}
```